Documentação do Projeto: Análise de Microdados ENEM

Nome: Leonardo Scholl Sternberg
Curso: Algoritmos E Programação
Especialização em Inteligência Artificial para Engenheiros

1: Introdução

Este relatório apresenta a documentação do código em Python ENEM.py, cujos dados são sobre a prova de 2024 do ENEM. O código se encontra em anexo, e os dados do ENEM estão disponíveis em: https://www.gov.br/inep/pt-br/acesso-a-informacao/dados-abertos/microdados/enem.

O enunciado do exercício a ser resolvido se encontra a seguir:
"Recupere os microdados e localize o arquivo que contém os resultados individuais de todos os inscritos no exame. Identifique, através da documentação e do próprio arquivo, as colunas que trazem os seguintes dados:
1) Notas das provas específicas e notas da redação
2) UF, Cidade e Escola de origem do aluno
3) Tipo de Escola (privada, pública, …)
Fazer uma carga parcial do arquivo, tratando apenas dessas colunas. Identifique dados inconsistentes ou faltantes. Produza dados agregados a partir dos dados brutos:
a) Média de notas por aluno
b) Média de notas por escola
c) Média de notas por município
d) Média de notas por UF

Produza um ranking dos 10 melhores alunos, 10 melhores escolas (geral e nos extratos públicos e privado) e 10 melhores municípios do país e do Rio Grande do Sul.

Posteriormente, extraia os dados em arquivos formato '.csv'. Selecione recortes dos dados que possam exemplificar os tipos de gráficos que trabalhamos (pelo menos 4 diferentes). Por exemplo:

- Histograma das diferentes provas para um dado conjunto populacional
- Gráficos de barra com os desempenhos de algumas cidades de uma dada unidade federada
- Gráfico waffle mostrando os recortes em uma dada capital dos desempenhos médios".

2: Análise dos Dados

2.1:Estrutura dos dados

A base de dados desejada do ENEM se encontra no arquivo 'DADOS\RESULTADOS_2024.csv'. Utilizando o dicionário de dados em 'DICIONÁRIO\Dicionário_Microdados_Enem_2024.xlsx', conseguimos responder os primeiros pontos requisitados.

1) Notas das provas específicas e notas da redação: ('NU_NOTA_CN', 'NU_NOTA_CH', 'NU_NOTA_LC', 'NU_NOTA_MT', 'NU_NOTA_REDACAO')

2) UF, Cidade e Escola de origem do aluno: ('SG_UF_ESC', 'NO_MUNICIPIO_ESC' e 'CO_ESCOLA')

3) Tipo de Escola (privada, pública, ...): ('TP_DEPENDENCIA_ADM_ESC')

2.2:Bibliotecas

Para o código, usou-se as seguintes bibliotecas:

- **Pandas:** É a ferramenta principal para manipulação e análise de dados, permitindo trabalhar com tabelas (DataFrames) de forma eficiente. Cálculos matemáticos também foram feitos utilizando ela ao invés de importar diretamente uma biblioteca matemática como o Numpy.
- **Seaborn:** Construída sobre o Matplotlib, é focada em visualizações estatísticas atraentes, informativas e fáceis de configurar.
- **Matplotlib:** É a biblioteca fundamental para criação de gráficos em Python, oferecendo controle total sobre cada elemento visual da imagem.
- **Pywaffle:** É uma extensão do Matplotlib projetada especificamente para a criação de gráficos de waffle.

2.3:Arquivos saída

Os resultados das medias e rankings foram extraídos em arquivos '.csv', nos seguintes arquivos

a) Média de notas por aluno (media_aluno.csv')

b) Média de notas por escola ('media_por_escola.csv')

c) Média de notas por município ('media_por_municipio.csv')

d) Média de notas por UF ('media_por_uf.csv')

e) Ranking das 10 melhores notas por aluno ('ranking_alunos.csv')

f) Ranking das 10 melhores notas por escola ('ranking_escolas.csv')

g) Ranking das 10 melhores notas por município ('ranking_municipios.csv')

h) Ranking das 10 melhores notas por UF ('ranking_municipios_rs.csv')

2.4:Gráficos de saída

Como gráficos de saída, foram gerados os seguintes gráficos:

- Histograma das Provas
- Gráfico de Barras: Desempenho de Cidades do Brasil
- Gráfico de Pizza: Proporção de Tipos de Escola em Porto Alegre
- Gráfico de Waffer: Proporção de Tipos de Escola em Porto Alegre
- Boxplot: Comparação de Notas por Tipo de Dependência Administrativa

3: Implementação e Avaliação do Código

Nas linhas a seguir, estão os trechos de códigos e explicações sobre o que foi realizado.

1: Importação do Pandas, leitura do arquivo CSV (somente colunas de interesse para otimização de memória), e tratamento de dados retirando linhas com dados faltantes.

```
# Carregando os dados
import pandas as pd
df = pd.read_csv('./RESULTADOS_2024.csv', encoding="iso-8859-1", sep=";", usecols=['NU_SEQUENCIAL', 'CO_ESCOLA', 'SG_UF_ESC', 'NO_MUNICIPIO_ESC',
'TP_DEPENDENCIA_ADM_ESC', 'NU_NOTA_CN', 'NU_NOTA_CH', 'NU_NOTA_LC', 'NU_NOTA_MT', 'NU_NOTA_REDACAO'])

# Tirando os dados faltantes
df = df.dropna(subset=['NU_NOTA_CN', 'NU_NOTA_CH', 'NU_NOTA_LC', 'NU_NOTA_MT', 'NU_NOTA_REDACAO'], how='all')
```

2: Calculando a media de notas por aluno (utilizando todas as colunas de notas). Posteriormente printando 5 linhas para visualização do conjunto resposta. Fazendo o mesmo para media das notas por escola, por município e por UF.

```
# Media de notas por aluno
df['media_aluno'] = df[['NU_NOTA_CN', 'NU_NOTA_CH', 'NU_NOTA_LC', 'NU_NOTA_MT', 'NU_NOTA_REDACAO']].mean(axis=1)
print("Média de notas por aluno:")
print(df[['NU_SEQUENCIAL', 'media_aluno']].head())
print("=" * 50)

# Media de notas por escola
media_por_escola = df.groupby('CO_ESCOLA')['media_aluno'].mean()
print("Média de notas por escola:")
print(media_por_escola.head())
print("=" * 50)

# Media de notas por município
media_por_municipio = df.groupby('NO_MUNICIPIO_ESC')['media_aluno'].mean()
print("Média de notas por município:")
print(media_por_municipio.head())
print("=" * 50)

# Media de notas por UF
media_por_uf = df.groupby('SG_UF_ESC')['media_aluno'].mean()
print("Média de notas por UF:")
print(media_por_uf.head())
print("=" * 50)
```

3: Obtendo o ranking dos 10 melhores alunos, utilizando a variável media_aluno. Depois obtendo o ranking das 10 melhores escolas utilizando a variável media_por_escola.

```
# Ranking dos 10 melhores alunos
ranking_alunos = df.sort_values(by='media_aluno', ascending=False).head(10)
print("Ranking dos 10 melhores alunos:")
print(ranking_alunos[['NU_SEQUENCIAL', 'media_aluno']])
print("=" * 50)

# Ranking das 10 melhores escolas
ranking_escolas = media_por_escola.sort_values(ascending=False).head(10)
print("Ranking das 10 melhores escolas:")
print(ranking_escolas)
print("=" * 50)
```

4: Obtendo o ranking das 10 melhores escolas de cada tipo ('TP_DEPENDENCIA_ADM_ESC' igual a 1 para federal, 2 para estadual, 3 para municipal e 4 para privada). Para isto, foi utilizado a variável media_aluno.

```
# Ranking das 10 melhores escolas federais
ranking_escolas_federais = df[df['TP_DEPENDENCIA_ADM_ESC'] == 1].groupby('CO_ESCOLA')['media_aluno'].mean().
sort_values(ascending=False).head(10)
print("Ranking das 10 melhores escolas federais:")
print(ranking_escolas_federais)
print("=" * 50)

# Ranking das 10 melhores escolas estaduais
ranking_escolas_estaduais = df[df['TP_DEPENDENCIA_ADM_ESC'] == 2].groupby('CO_ESCOLA')['media_aluno'].mean().
sort_values(ascending=False).head(10)
print("Ranking das 10 melhores escolas estaduais:")
print(ranking_escolas_estaduais)
print("=" * 50)

# Ranking das 10 melhores escolas municipais
ranking_escolas_municipais =df[df['TP_DEPENDENCIA_ADM_ESC'] == 3].groupby('CO_ESCOLA')['media_aluno'].mean().
sort_values(ascending=False).head(10)
print("Ranking das 10 melhores escolas municipais:")
print(ranking_escolas_municipais)
print("=" * 50)

# Ranking das 10 melhores escolas privadas
ranking_escolas_privadas = df[df['TP_DEPENDENCIA_ADM_ESC'] == 4].groupby('CO_ESCOLA')['media_aluno'].mean().
sort_values(ascending=False).head(10)
print("Ranking das 10 melhores escolas privadas:")
print(ranking_escolas_privadas)
print("=" * 50)
```

5: Obtendo o ranking dos 10 melhores municípios do país, e dos 10 melhores municípios do Rio Grande do Sul. Para isto, foi utilizado a variável media_aluno.

```
# Ranking dos 10 melhores municípios do país
ranking_municipios = media_por_municipio.sort_values(ascending=False).head(10)
print("Ranking dos 10 melhores municípios do país:")
print(ranking_municipios)
print("=" * 50)

# Ranking dos 10 melhores municípios do Rio grande do Sul
ranking_municipios_rs = df[df['SG_UF_ESC'] == 'RS'].groupby('NO_MUNICIPIO_ESC')['media_aluno'].mean().sort_values
(ascending=False).head(10)
print("Ranking dos 10 melhores municípios do Rio Grande do Sul:")
print(ranking_municipios_rs)
```

6: Exportando os resultados em arquivo CSV.

```
# Colocando os conjuntos respostas em csv
df['media_aluno'].to_csv('media_aluno.csv', index=False)
media_por_escola.to_csv('media_por_escola.csv', index=False)
media_por_municipio.to_csv('media_por_municipio.csv', index=False)
media_por_uf.to_csv('media_por_uf.csv', index=False)
ranking_alunos.to_csv('ranking_alunos.csv', index=False)
ranking_escolas.to_csv('ranking_escolas.csv', index=True)
ranking_municipios.to_csv('ranking_municipios.csv', index=True)
ranking_municipios_rs.to_csv('ranking_municipios_rs.csv', index=True)
```

7: Importação de bibliotecas para poder plotar gráficos e configuração estética.

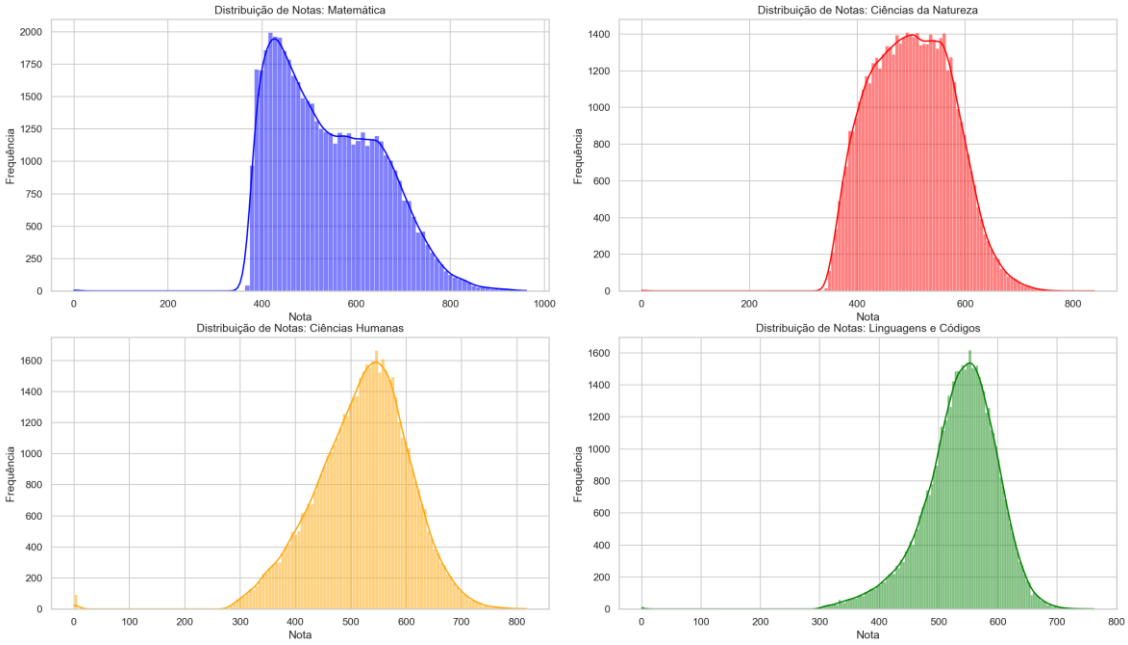
```
import matplotlib.pyplot as plt
import seaborn as sns
from pywaffle import Waffle

# Configuração estética geral
sns.set_theme(style="whitegrid")
```

8: Plotando gráfico histograma com distribuições dos resultados das quatro provas (matriz 2x2).

```
# --- 1. Histograma das Provas ---
df_rs = df[df['SG_UF_ESC'] == 'RS']

# Criar uma figura com 2 linhas e 2 colunas
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
configuracoes = [
    ('NU_NOTA_MT', 'Matemática', 'blue', axes[0, 0]),
    ('NU_NOTA_CN', 'Ciências da Natureza', 'red', axes[0, 1]),
    ('NU_NOTA_CH', 'Ciências Humanas', 'orange', axes[1, 0]),
    ('NU_NOTA_LC', 'Linguagens e Códigos', 'green', axes[1, 1])
]
for coluna, titulo, cor, ax in configuracoes:
    sns.histplot(df_rs[coluna], kde=True, color=cor, ax=ax)
    ax.set_title(f'Distribuição de Notas: {titulo}')
    ax.set_xlabel('Nota')
    ax.set_ylabel('Frequência')
plt.tight_layout()
plt.show()
```



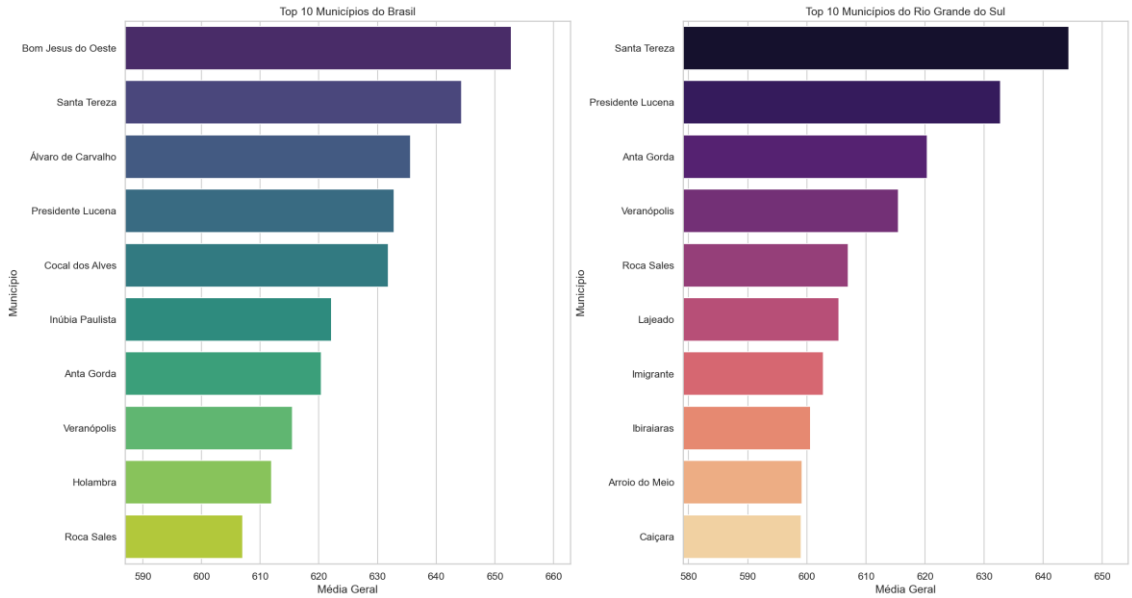
9: Plotando gráfico de barra do desempenho das melhores cidades do Brasil e das melhores cidades do Rio Grande do Sul (matriz 1x2).

```
# --- 2. Gráfico de Barras: Desempenho de Cidades do Brasil ---
fig, axes = plt.subplots(1, 2, figsize=(18, 7))

# Grafico da esquerda
sns.barplot(
    x=ranking_municipios.values,
    y=ranking_municipios.index,
    ax=axes[0],
    palette="viridis"
)
axes[0].set_title('Top 10 Municípios do Brasil')
axes[0].set_xlabel('Média Geral')
axes[0].set_ylabel('Município')
# Ajuste de escala dinâmica para destacar as diferenças
axes[0].set_xlim(min(ranking_municipios.values) - 20, max(ranking_municipios.values) + 10)

# Grafico da direita
sns.barplot(
    x=ranking_municipios_rs.values,
    y=ranking_municipios_rs.index,
    ax=axes[1],
    palette="magma"
)
axes[1].set_title('Top 10 Municípios do Rio Grande do Sul')
axes[1].set_xlabel('Média Geral')
axes[1].set_ylabel('Município')
# Ajuste de escala dinâmica para destacar as diferenças
axes[1].set_xlim(min(ranking_municipios_rs.values) - 20, max(ranking_municipios_rs.values) + 10)

plt.tight_layout()
plt.show()
```

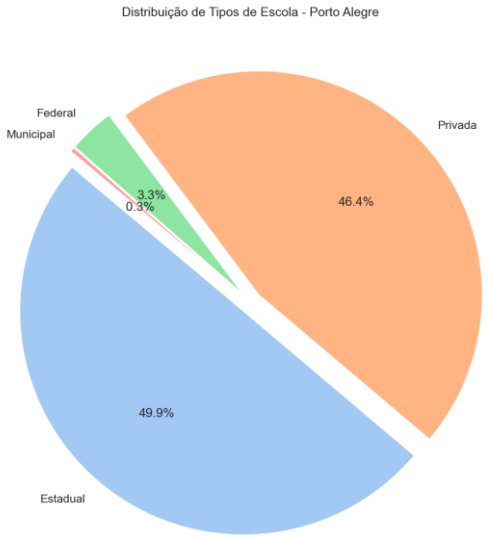


10: Plotando gráfico de pizza da distribuição dos tipos de escola em Porto Alegre.

```
# --- 3. Gráfico de Pizza: Proporção de Tipos de Escola em Porto Alegre ---
contagem_escolas = df[df['NO_MUNICIPIO_ESC'] == 'Porto Alegre']['TP_DEPENDENCIA_ADM_ESC'].value_counts()
legenda_map = {1: 'Federal', 2: 'Estadual', 3: 'Municipal', 4: 'Privada'}
labels = [legenda_map[k] for k in contagem_escolas.index]
valores = contagem_escolas.values

plt.figure(figsize=(8, 8))
plt.pie(
    valores,
    labels=labels,
    autopct='%1.1f%%', # Mostra a porcentagem com uma casa decimal
    startangle=140,
    colors=sns.color_palette("pastel"),
    explode=[0.05] * len(labels) # Afasta levemente as fatias para facilitar a leitura
)

plt.title('Distribuição de Tipos de Escola - Porto Alegre')
plt.show()
```

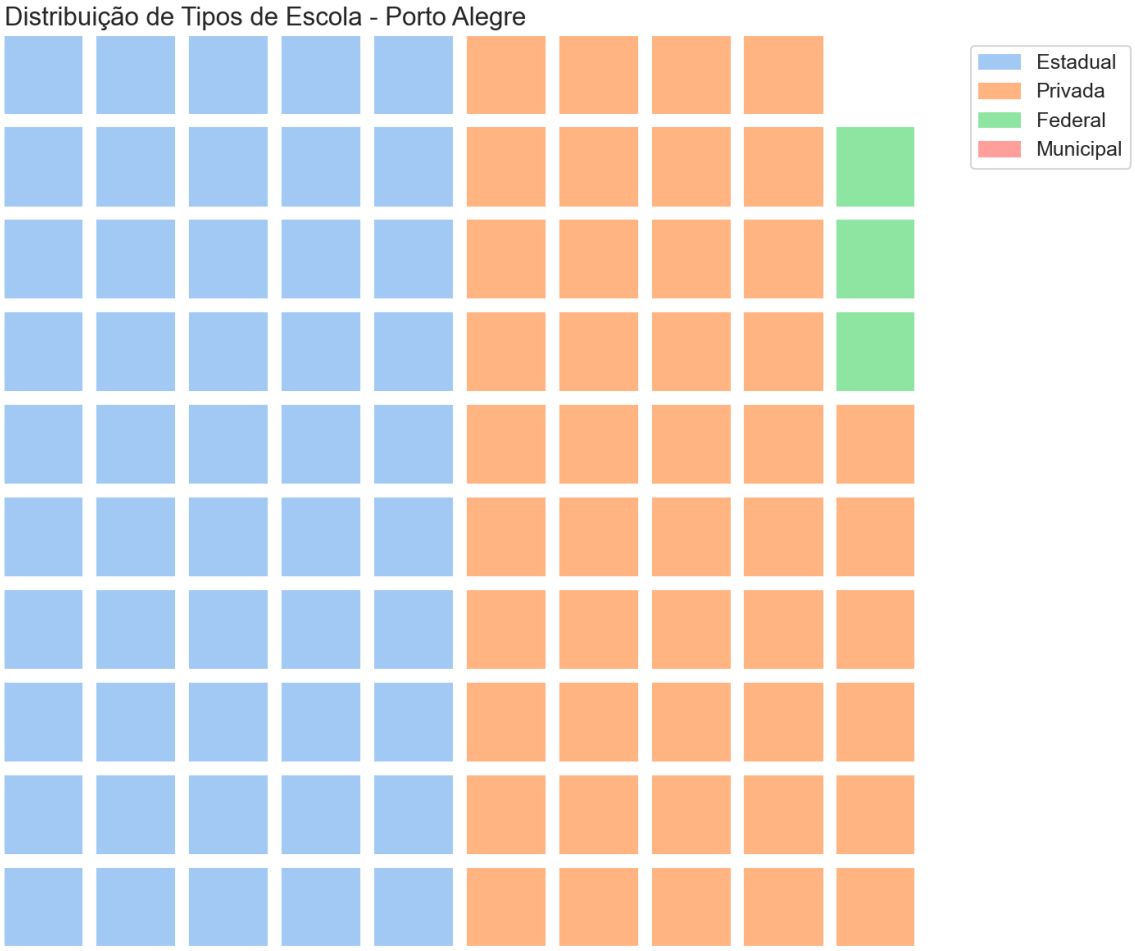


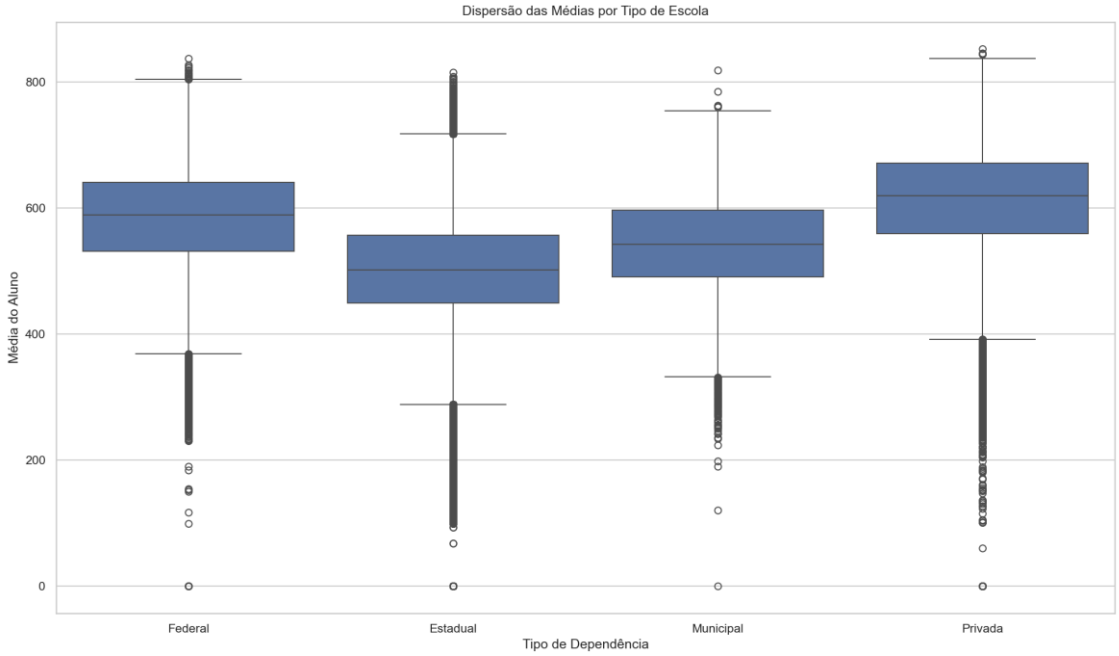
11: Plotando gráfico de Waffer da distribuição dos tipos de escola em Porto Alegre. Mesmos dados que o anterior, mas plotado de outra forma.

```
# --- 4. Gráfico de Waffers: Proporção de Tipos de Escola em Porto Alegre ---
contagem_escolas = df[df['NO_MUNICIPIO_ESC'] == 'Porto Alegre']['TP_DEPENDENCIA_ADM_ESC'].value_counts()
legenda_map = {1: 'Federal', 2: 'Estadual', 3: 'Municipal', 4: 'Privada'}

data_waffle = {legenda_map[k]: v for k, v in contagem_escolas.items()}
cores_dinamicas = sns.color_palette("pastel", len(data_waffle)).as_hex()

fig = plt.figure(
    FigureClass=Waffle,
    rows=10,
    columns=10,
    values=data_waffle,
    colors=cores_dinamicas,
    title={'label': 'Distribuição de Tipos de Escola - Porto Alegre', 'loc': 'left', 'fontsize': 15},
    legend={'loc': 'upper left', 'bbox_to_anchor': (1.05, 1), 'fontsize': 12},
    starting_location='NW', # Começa a pintar do canto superior esquerdo
    block_arranging_style='snake', # Estilo de preenchimento (opcional)
    figsize=(8, 8)
)
plt.show()
```



12: Plotando gráfico boxplot para comparação de notas por tipo de dependência administrativa. A linha central representa a mediana. O corpo da caixa representa o intervalo interquartil. As hastes mostram a variabilidade fora da caixa (limite superior = Q3 + (1,5 * IQR); limite inferior = Q1 - (1,5 * IQR)). Os pontos além das hastes representam outliers.

```
# --- 5. Boxplot: Comparação de Notas por Tipo de Dependência Administrativa ---
plt.figure(figsize=(10, 6))
sns.boxplot(x='TP_DEPENDENCIA_ADM_ESC', y='media_aluno', data=df)
plt.xticks([0, 1, 2, 3], ['Federal', 'Estadual', 'Municipal', 'Privada'])
plt.title('Dispersão das Médias por Tipo de Escola')
plt.ylabel('Média do Aluno')
plt.xlabel('Tipo de Dependência')
plt.show()
```